

through a tiny push-to-on switch as shown in Fig. 3.

Working of the project is simple. Once the program is running on Raspberry Pi (explained in the software section), press the push-to-on button once and see the message changing from "Button is released" to "Button is pressed" as shown in Fig. 1. So now a physical command has been entered and deciphered by Raspberry Pi.

Software. To run the software program designed in PHP successfully, you need PHP and PHP-GPIO libraries installed in your system.

PHP can be easily installed by the following command:

```
sudo apt-get install php5
```

This command runs without any issues and eventually comes back to the prompt. Once it completes, install the PHP-GPIO libraries to interact with the GPIO. Download the libraries from:

<https://github.com/aaronpearce/PHP-GPIO>.

Extract libraries and paste them into '/var/www/' (take care that you paste only files, not the folder itself).

Now open the terminal window on the Raspberry Pi screen and enter the following command to open the nano text editor:

```
nano testinput.php
```

where 'nano' is the text editor and 'testinput' is the name of the project that we have given.

Paste the below-mentioned program in the nano text editor:

```
#!/usr/bin/php
<?php
require_once('GPIO.php');
$button = 18; // connect to physical
pin no 12 which is GPIO 18
$gpio = new GPIO();
$gpio->setup($button, "in"); // set
the button for data input mode
while(true) { // beginning of an
infinite loop
$val = $gpio->input($button); // read
the state of the button
//usleep(0.5 * 1000000); // 0.5 second
delay timer ; uncomment it for use
if ($val == 0) echo "Button is
```

```
pressed \n";
else echo "Button is released \n";
}
?>
```

Save it by pressing 'control+o' and then exit by 'control+x.' Move this file to '/var/www/' from where it is executed (you need to be a sudo user for that).

Now to execute the code, issue the following command in the terminal:

```
sudo php -f /var/www/testinput.php
```

You will see the terminal converted to something as shown in Fig. 1, indicating "Button is released" by default. Now if you press the button, the status will change to "Button is pressed."

Project 2: Switching combinations

Now that you have learnt how to read inputs from GPIO, you can move one step further by triggering some events corresponding to the inputs. In this project, for each press of the push-to-on switch, different LEDs will switch on and blink indefinitely.

Circuit and working. Fig. 4 shows the circuit for this project. The circuit of the previous project is used here with

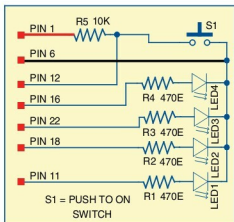


Fig. 4: Circuit for switching combinations

some additional wiring. Here we control four LEDs, all connected to four different GPIOs at pins 11, 18, 22 and 16. The push-to-on switch is connected at GPIO-18, which is available at physical pin 12. The on-board 3.3 volts is taken from pin 1 through a 10-kilo-ohm resistor (any value between 3.3 and 10 kilo-ohms will work) and the ground is taken from pin 6.



Raspberry Pi Camera

High-Definition video camera for Raspberry Pi



New Launch

Raspberry Pi Camera
Rev 1.0 KOL0413

RS Stock No. 775-7731

Raspberry Pi

Model A



RS Stock No. 756-0317
Order now for just ₹ 2175/-

Model B



RS Stock No. 756-0308
Order now for just ₹ 2950/-

Available on Next Day Delivery

www.rsindia.com
Toll Free: 1800-180-7746



RS Components & Controls (India) Ltd.

*Taxes extra



DESIGNSPARK

Find out more useful resources on Raspberry Pi at,

www.designspark.com